Project Report: Linear Reconstruction for the ATLAS Tile Calorimeter at the HL-LHC

Prethiv Sriman D
S.R.M. Institute of Science and Technology

Google Summer of Code (GSoC' 26)
Organization: **CERN-HSF**

Mentors: Luca Fiorini & Fernando Carriò

March 12, 2026

Abstract

This report details the development of a ridge regression model designed to estimate the Low-Gain (LG) channel energy from 7-sample ADC pulse windows recorded by the ATLAS Tile Calorimeter. The model incorporates L_2 regularization via the AdamW optimizer to suppress overfitting to noisy pulse wings, and predicts the reconstructed energy through a single weighted sum of the input samples, satisfying the strict linearity requirements of FPGA-based hardware trigger systems. Performance is evaluated on a held-out test set using MSE, MAE, R^2, mean relative error, and RMS relative error, achieving an R^2 of 0.9906 and a mean relative error of 0.41.

Contents

1 Introduction **3**

2 Problem Statement **3**
2.1 The Nature of the Signal . . . 3
2.2 Technical Challenges . . . 3
2.3 Algorithmic Constraints . . . 3

3 Dataset Description **4**
3.1 Dataset Structure . . . 4
3.2 Input Representation and Sample Timing . . . 4
3.3 Dataset Splits . . . 5
3.4 Gain Channels . . . 5
3.5 Target Variable . . . 5
3.6 Optimal Filter Reference . . . 5
3.7 Data Loading . . . 6

4 Methodology: Linear Reconstruction **6**
4.1 Model Architecture . . . 6
4.2 Model 1: Baseline Linear Regression . . . 6
4.3 Model 2: Ridge Regression . . . 7
4.4 Linearity and Hardware Compatibility . . . 7

5 Implementation Details **7**
5.1 Framework and Infrastructure . . . 7
5.2 Training Configuration . . . 8
5.3 Training Loop . . . 8
5.4 Closed-Form Equivalence . . . 8

6 Results & Evaluation **8**
6.1 Evaluation Procedure . . . 8
6.2 Performance Metrics . . . 8
6.3 Quantitative Results . . . 9
6.4 Interpretation of Relative Error Metrics . . . 9
6.5 Training and Validation Loss Curves . . . 10
6.6 Relative Error Distribution . . . 11
6.7 Relative Error vs. True Energy . . . 11
6.8 Absolute Error vs. True Energy . . . 13

7 Conclusion **13**

1 Introduction

The ATLAS Tile Calorimeter (TileCal) is the central hadronic calorimeter of the ATLAS experiment at the Large Hadron Collider (LHC). Its primary function is to measure the energy of hadronic jets and contribute to the reconstruction of missing transverse energy. The detector is a sampling calorimeter utilizing steel as an absorber and plastic scintillators as the active medium. The light produced by charged particles in the scintillators is transmitted via wavelength-shifting fibers to photomultiplier tubes (PMTs).

As the LHC transitions into the High-Luminosity (HL-LHC) era, the calorimeter will face significantly higher luminosity and data rates. PMT signals are digitized by 12-bit Analog-to-Digital Converters (ADCs) every 25 ns, corresponding to the LHC bunch-crossing frequency. The Phase-II upgrade requires real-time, bunch-crossing-wise energy reconstruction performed directly on FPGAs with extremely low latency (approximately 225 ns).

Historically, TileCal has relied on a linear Optimal Filtering (OF) algorithm, which uses a weighted sum of digital samples to estimate signal amplitude and time while minimizing noise. However, increased pile-up, where pulses from consecutive bunch crossings overlap, creates non-Gaussian noise components that challenge traditional linear filters.

2 Problem Statement

The core objective of this project is to develop and compare robust linear algorithms capable of accurately reconstructing the Low-Gain channel target energy from a sequence of noisy, normalized digital samples.

2.1 The Nature of the Signal

A physical signal in the TileCal is characterized by three primary parameters:

- **Pedestal:** The baseline ADC count in the absence of a signal.
- **Amplitude:** A value proportional to the deposited energy.
- **Phase:** The time shift of the pulse peak relative to the nominal bunch crossing.

2.2 Technical Challenges

The reconstruction process must overcome three significant hurdles:

- **Electronic Noise:** Intrinsic fluctuations in the readout electronics that degrade energy resolution, particularly at low energies.
- **Pedestal Fluctuations:** The baseline is not fixed; it is affected by both electronic noise and nearby signal activity.
- **Pile-up Interference:** At HL-LHC luminosities, signals from a specific bunch crossing are frequently contaminated by "out-of-time" pulses from previous or subsequent crossings.

2.3 Algorithmic Constraints

To meet the stringent hardware requirements of the ATLAS Level-0 trigger system, the algorithm must be fully linear. For each bunch crossing n, the model maps a fixed window of 7 input digital samples covering $[n-3, n+3]$ to a single predicted energy value:

$$\hat{y} = \sum_{i=0}^{6} w_i \, x_i + b \tag{1}$$

This linearity ensures the algorithm can be implemented as a simple high-speed weighted sum on FPGAs, without the computational overhead of non-linear activation functions.

3 Dataset Description

The dataset used in this work is stored as PyTorch shard files (`.pt`) containing preprocessed detector simulations from the ATLAS Tile Calorimeter. Each shard contains 2048 detector events stored as PyTorch tensors. The dataset is divided into training, validation, and testing subsets to ensure proper model evaluation and generalization.

3.1 Dataset Structure

Each `.pt` shard contains three tensors:

- **Input Samples (X)**
- **Target Labels (y)**
- **Optimal Filter Estimates (y_{OF})**

The tensor dimensions within each shard are:

Tensor	Shape	Datatype	Description
X	$[2048, 2, 7]$	float32	ADC pulse samples (both gain channels)
y	$[2048, 2]$	float32	Ground truth energy targets
y_{OF}	$[2048, 2]$	float32	Optimal Filter reconstruction

Table 1: Tensor structure contained in each PyTorch shard.

Field	Physical Label	Scaling/State
$X[:, 0, :]$	High-Gain (HG) ADC Samples	Normalized
$X[:, 1, :]$	Low-Gain (LG) ADC Samples	Normalized
$y[:, 0]$	E_{true} (HG channel)	Scaled Physics Target
$y[:, 1]$	E_{true} (LG channel)	Scaled Physics Target
$y_{OF}[:, 0]$	HG Optimal Filter Energy	Raw ADC Counts
$y_{OF}[:, 1]$	LG Optimal Filter Energy	Raw ADC Counts

Table 2: Physical labels and scaling state for each tensor field. This work uses $X[:, 1, :]$ as input and $y[:, 1]$ as the regression target.

Each shard contains **2048 events**. The dataset is stored as multiple shard files to enable efficient streaming and loading during training.

3.2 Input Representation and Sample Timing

The full input tensor X has the shape:

$$X \in \mathbb{R}^{2048 \times 2 \times 7}$$

where 2048 represents the number of events in a shard, 2 corresponds to the two gain channels, and 7 represents the digitized ADC samples of the pulse. In this work, only the **Low-Gain channel** ($X[:, 1, :]$) is used as the model input, yielding a per-event feature vector of shape $\mathbb{R}^7$.

The 7 samples span a window centred on the in-time bunch crossing n, covering the range $[n - 3, n + 3]$. Table 3 details the physical interpretation of each sample index.

Time Index	Sample Mapping in X	Physical Context
$n-3$	Sample 1	75 ns before collision
$n-2$	Sample 2	50 ns before collision (Out-of-time pile-up zone)
$n-1$	Sample 3	25 ns before collision
n	**Sample 4**	**In-time collision sample (Pulse Peak)**
$n+1$	Sample 5	25 ns after collision
$n+2$	Sample 6	50 ns after collision (Pulse tail/pile-up zone)
$n+3$	Sample 7	75 ns after collision

Table 3: Physical timing context for each of the 7 ADC samples in the input window. Sample 4 (index n) corresponds to the in-time collision and carries the peak signal amplitude.

3.3 Dataset Splits

The full dataset was divided into three partitions:

Split	Number of Shards	Approximate Events
Training	261	∼534,528
Validation	56	∼114,688
Testing	56	∼114,688

Table 4: Dataset partitioning used for training and evaluation.

3.4 Gain Channels

TileCal uses a dual-gain readout system to preserve signal quality across a wide dynamic range:

- **High-Gain (HG):** Sensitive to small energy deposits but saturates for large signals.
- **Low-Gain (LG):** Handles large energy deposits without saturation but has lower resolution for small signals.

In this work, all models are trained and evaluated exclusively on the **Low-Gain channel**, using $X[:,1,:]$ as input features and $y[:,1]$ as the reconstruction target.

3.5 Target Variable

The scalar regression target for each event is $y[:,1]$, corresponding to the scaled Low-Gain true energy:

$$y_{\text{LG}} \in \mathbb{R}^{N_{\text{events}}}$$

During evaluation, predictions and ground-truth values are denormalized using the dataset statistics (mean and standard deviation) to recover physical energy units for relative error computation.

3.6 Optimal Filter Reference

The tensor y_{OF} contains the output of the traditional **Optimal Filtering (OF)** algorithm currently used in the ATLAS TileCal reconstruction pipeline. Optimal Filtering reconstructs the signal amplitude using a weighted linear combination of ADC samples:

$$A = \sum_{i=1}^{N} w_i s_i \tag{2}$$

where s_i are the ADC samples and w_i are the optimal filter weights derived from the known pulse shape and noise covariance. Although y_{OF} is available in the dataset, a quantitative comparison against the OF baseline is reserved for future work.

3.7 Data Loading

During training, shards are loaded using PyTorch's `Dataset` and `DataLoader` utilities. All shard files are read into memory at initialization, with the High-Gain samples (X_{hi}), Low-Gain samples (X_{lo}), and LG targets (y) extracted and stored as separate tensors. DataLoaders are configured with a batch size of 1024 for training (with shuffling) and 2048 for validation and testing.

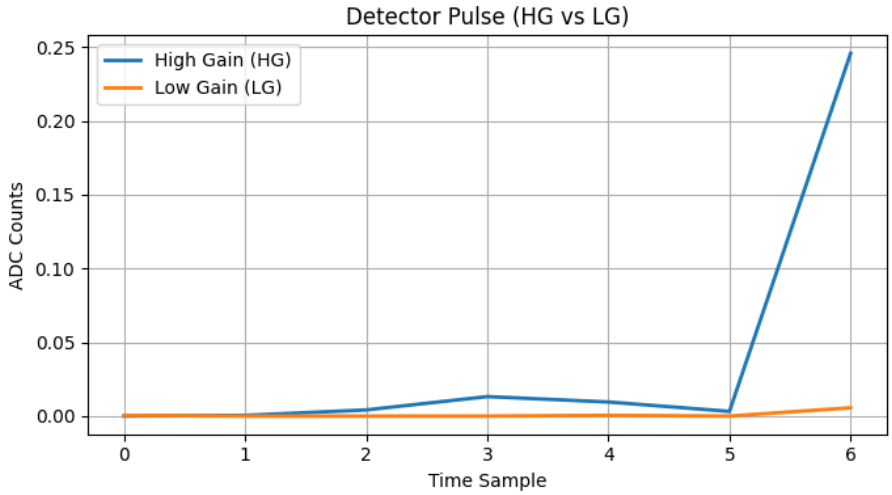

Figure 1: Visualization of a single detector pulse showcasing the High-Gain and Low-Gain channel ADC counts.

4 Methodology: Linear Reconstruction

To adhere strictly to the latency requirements of FPGA-based hardware trigger systems, both models are designed to be completely linear – each produces its output as a single weighted sum of the 7 input samples with no non-linear transformations.

4.1 Model Architecture

Both models share the same `LinearRegression` architecture, implemented as a single `nn.Linear(7, 1)` layer in PyTorch:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad \mathbf{x} \in \mathbb{R}^7,\ \hat{y} \in \mathbb{R} \tag{3}$$

where $\mathbf{w} \in \mathbb{R}^7$ are the learned sample weights and b is the learned bias. The two models differ only in their optimizer and regularization strategy.

4.2 Model 1: Baseline Linear Regression

The first model is a standard linear regression trained to minimize the Mean Squared Error (MSE) loss using the Adam optimizer:

$$\mathcal{L}_{\text{LR}} = \frac{1}{M} \sum_{j=1}^{M} (y_j - \hat{y}_j)^2 \tag{4}$$

This model serves as the unregularized baseline. It is trained for 100 epochs with a learning rate of 1×10^{-3} and a batch size of 1024.

4.3 Model 2: Ridge Regression

The second model adds L_2 regularization to the weights, implemented via the `AdamW` optimizer's built-in `weight_decay` parameter. The effective objective being minimized is:

$$\mathcal{L}_{\text{Ridge}} = \frac{1}{M}\sum_{j=1}^{M}(y_j - \hat{y}_j)^2 + \lambda\|\mathbf{w}\|_2^2 \tag{5}$$

where $\lambda = 1 \times 10^{-3}$ is the regularization strength. This regularization discourages the model from relying heavily on individual noisy samples in the pulse wings, improving stability under pile-up conditions. This model is trained for 120 epochs with a learning rate of 9×10^{-3}.

4.4 Linearity and Hardware Compatibility

Both models satisfy the "linear algorithm" constraint: at inference time, each prediction reduces to a single dot product plus a bias term. The 7 learned weights $\mathbf{w}$ act analogously to Optimal Filter coefficients and can be directly stored as fixed-point integers on an FPGA, requiring no multipliers beyond a single MAC (multiply-accumulate) array.

5 Implementation Details

5.1 Framework and Infrastructure

The training pipeline was implemented in Python using PyTorch. All shard files are loaded and concatenated at dataset initialization. The `TileCalDataset` class exposes three items per event: the High-Gain 7-sample vector X_{hi}, the Low-Gain 7-sample vector X_{lo}, and the scalar LG target y. Only X_{lo} and y are used during training and evaluation in this work.

```python
class TileCalDataset(Dataset):
    def __init__(self, dataDir, norm=False):
        self.files = sorted(glob.glob(f'{dataDir}/*.pt'))
        li_X, li_y = [], []

        for file in self.files:
            data = torch.load(file)
            li_X.append(data['X'].float())
            li_y.append(data['y'].float())

        X = torch.cat(li_X, dim=0)
        y = torch.cat(li_y, dim=0)

        self.X_hi = X[:, 0, :]    # High-Gain channel
        self.X_lo = X[:, 1, :]    # Low-Gain channel
        self.y    = y[:, 1]        # LG energy target

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X_hi[idx], self.X_lo[idx], self.y[idx]
```

Listing 1: Dataset class for loading and preprocessing TileCal shard files.

```python
class LinearRegression(nn.Module):
    def __init__(self, in_dim):
        super().__init__()
        self.linear = nn.Linear(in_dim, 1)

    def forward(self, X):
        return self.linear(X)
```

Listing 2: Linear model architecture shared by both the baseline and ridge regression models.

Hyperparameter	Baseline Linear Regression	Ridge Regression
Optimizer	Adam	AdamW
Learning Rate	1×10^{-3}	9×10^{-3}
Regularization λ	None	1×10^{-3}
Epochs	100	120
Batch Size	1024	1024
Loss Function	MSE	MSE

Table 5: Hyperparameter configuration for the two trained models.

5.2 Training Configuration

5.3 Training Loop

Both models are trained using a shared `train()` function that handles epoch iteration, batch processing, loss computation, backpropagation, and per-epoch validation. A Keras-style progress bar (`Progbar`) provides live loss monitoring. At the final epoch, validation predictions and ground-truth values are collected for inspection.

5.4 Closed-Form Equivalence

Note: The ridge regression weights can equivalently be computed in closed form as $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top y$*, which eliminates the need for iterative optimization. The gradient-based approach via AdamW was used here for consistency with the training framework.*

6 Results & Evaluation

6.1 Evaluation Procedure

Both models are evaluated on the held-out test set using the shared `evaluate()` function. Standard regression metrics are computed on the normalized predictions, after which both predictions and targets are denormalized using the dataset statistics:

$$y_{\text{physical}} = y_{\text{normalized}} \times \sigma_{\text{LG}} + \mu_{\text{LG}}$$

Relative error metrics are then computed in physical energy units. A small epsilon ($\varepsilon = 10^{-8}$) is added to the denominator to guard against division by near-zero values:

6.2 Performance Metrics

Model performance is quantified using the following five metrics. MSE, MAE, and R^2 are computed on normalized predictions, as normalization stabilizes optimization and prevents large-energy events from dominating the training loss. Mean and RMS relative error are computed after denormalization to physical energy units.

```python
eps = 1e-8

# Relative error per sample
relative_error = (y_pred - y_true) / (y_true + eps)

# Mean relative error (systematic bias)
mean_relative_error = np.mean(relative_error)

# RMS relative error (overall spread)
rms_relative_error = np.sqrt(np.mean(relative_error**2))
```

Listing 3: Relative error computation in physical energy units.

- **MSE:** $\frac{1}{N}\sum(\hat{y}_i - y_i)^2$, average squared deviation on normalized scale.
- **MAE:** $\frac{1}{N}\sum|\hat{y}_i - y_i|$, average absolute deviation, less sensitive to outliers.
- R^2 **Score:** Fraction of variance in the target energy explained by the model.
- **Mean Relative Error:** $\frac{1}{N}\sum\frac{\hat{y}_i - y_i}{y_i}$, measures systematic bias in physical units.
- **RMS Relative Error:** $\sqrt{\frac{1}{N}\sum\left(\frac{\hat{y}_i - y_i}{y_i}\right)^2}$, measures the overall spread of reconstruction errors.

6.3 Quantitative Results

Table 6 summarises the test-set performance of both models across all evaluated metrics.

Metric	Linear Regression	Ridge Regression
MSE	0.3240	**0.0111**
MAE	0.0632	**0.0302**
R^2 Score	0.7263	**0.9906**
Mean Relative Error	2.4496	**0.4122**
RMS Relative Error	30.3146	**10.0534**

Table 6: Test-set performance comparison between the Baseline Linear Regression and Ridge Regression models. Bold values indicate the better result for each metric.

The ridge regression model outperforms the baseline across every metric by a substantial margin. The R^2 score improves from 0.7263 to 0.9906, indicating that the L_2-regularized model explains over 99% of the variance in the LG target energy. The MSE drops by a factor of approximately 29×, and the RMS relative error falls from 30.3 to 10.1, reflecting significantly tighter reconstruction across the energy spectrum.

6.4 Interpretation of Relative Error Metrics

Despite the strong R^2 and low absolute errors, the mean and RMS relative errors appear elevated and warrant careful interpretation. These metrics are computed as $(\hat{y} - y)/y$, which makes them inherently sensitive to low-energy bunch crossings where y is small. Although these values appear large, relative error is highly sensitive to small denominators: even small absolute prediction errors can lead to large relative deviations when the true energy is low. This does not contradict the R^2 of 0.9906, which is computed on the normalized scale and reflects the model's strong overall reconstruction quality. A diagnostic inspection of the test set confirms this: while the

minimum true energy is 0.25 ADC counts (well above zero), over 50,000 samples have relative errors exceeding 100%, with a maximum relative error of 760× and a 99th percentile of 24.

At low true energies, overlapping pulses from neighbouring bunch crossings can completely obscure a small true signal, causing the model to predict a near-zero amplitude where a genuine low-energy deposit exists. This produces relative errors approaching -1 (i.e. $\hat{y} \approx 0$), visible as a diagonal band in the 2D relative error plot at low E_{true} values. The RMS relative error is particularly sensitive to this effect because squaring the large outlier errors amplifies their contribution dramatically. The RMS of 10.05 versus a mean of 0.41 is itself evidence that a small number of extreme outlier predictions, rather than systematic bias, are driving the metric.

This behaviour is consistent with results reported in the reference literature for MLP and CNN models trained on identical data under the same pile-up conditions, where the same diagonal artefact appears in the low-energy regime [1, 2]. The linear model performs well across the mid-to-high energy range where the signal-to-noise ratio is favourable.

6.5 Training and Validation Loss Curves

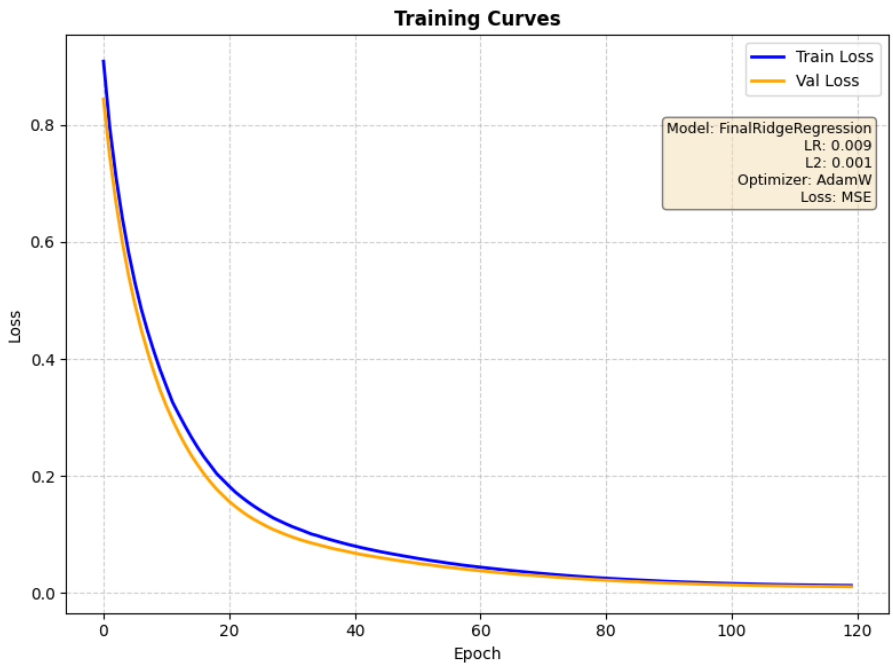

Figure 2: Training and validation loss curves (MSE vs. epoch) for the Ridge Regression model over 120 epochs.

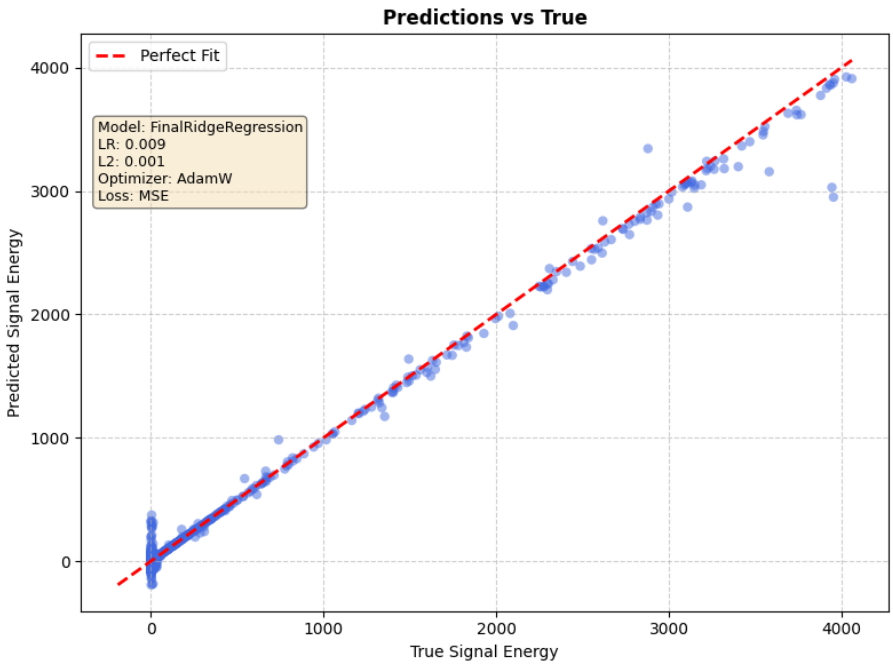

Figure 3: Predicted energy vs. true energy for the Ridge Regression model on the validation set. The dashed red line indicates the ideal $y = x$ reconstruction.

6.6 Relative Error Distribution

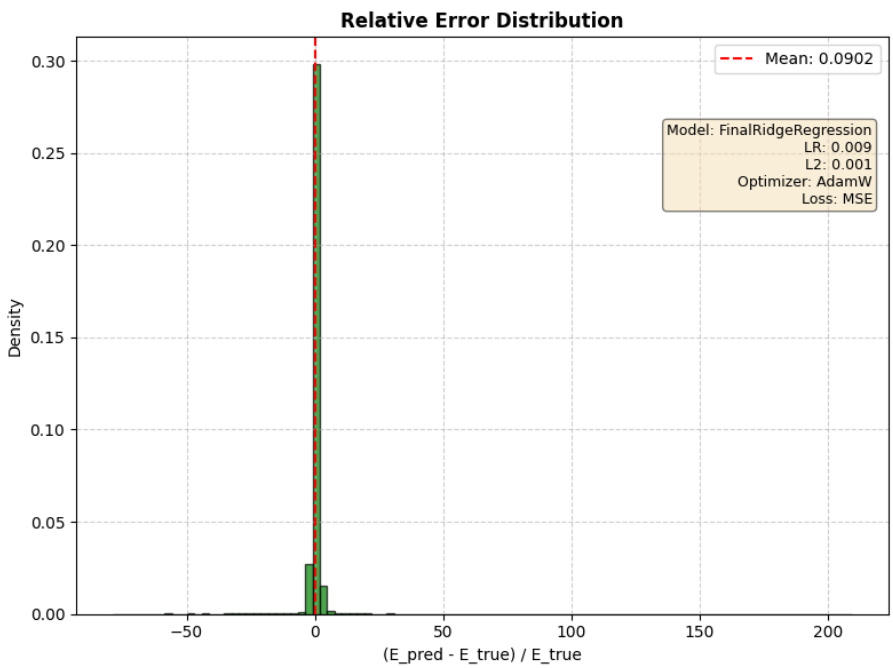

Figure 4: Distribution of the relative reconstruction error $(E_{\text{pred}} - E_{\text{true}})/E_{\text{true}}$ for the Ridge Regression model. The distribution is centred near zero, with heavy tails driven by low-energy events where pile-up conditions obscure the true signal. RMS relative error: 10.0534.

6.7 Relative Error vs. True Energy

Figure 5 shows the distribution of the relative reconstruction error $(E_{\text{pred}} - E_{\text{true}})/E_{\text{true}}$ as a function of the true signal energy. Since the majority of events lie in the low-energy region, a zoomed version of the plot is shown first to better visualise the error behaviour below 60 ADC counts. Figure 6 then shows the same distribution across the full dataset energy range.

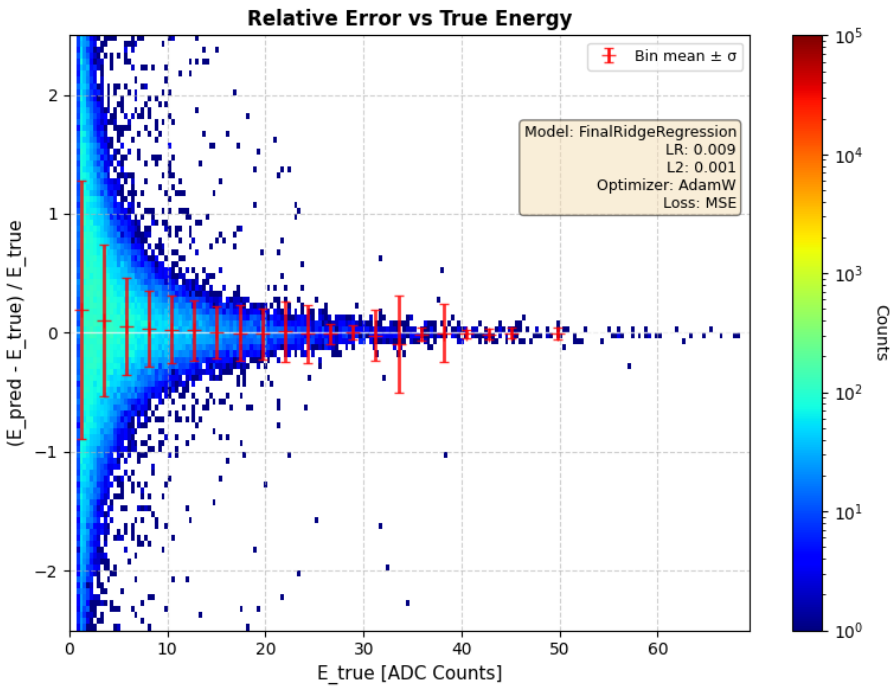

Figure 5: **Relative Error vs True Energy (Zoomed Low-Energy Region).** 2D distribution of the relative reconstruction error $(E_{\text{pred}} - E_{\text{true}})/E_{\text{true}}$ as a function of the true signal energy. The plot is zoomed to $E_{\text{true}} < 60$ ADC counts to highlight the behaviour in the low-energy region where most events are concentrated.

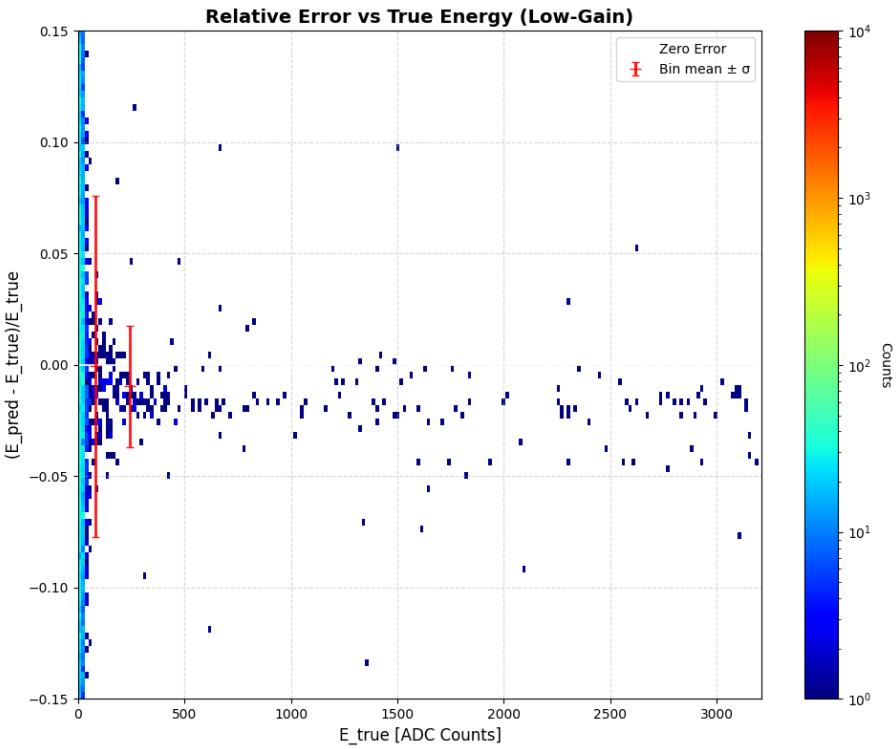

Figure 6: **Relative Error vs True Energy (Full Energy Range).** 2D distribution of the relative reconstruction error $(E_{\text{pred}} - E_{\text{true}})/E_{\text{true}}$ as a function of the true signal energy across the full dataset range.

6.8 Absolute Error vs. True Energy

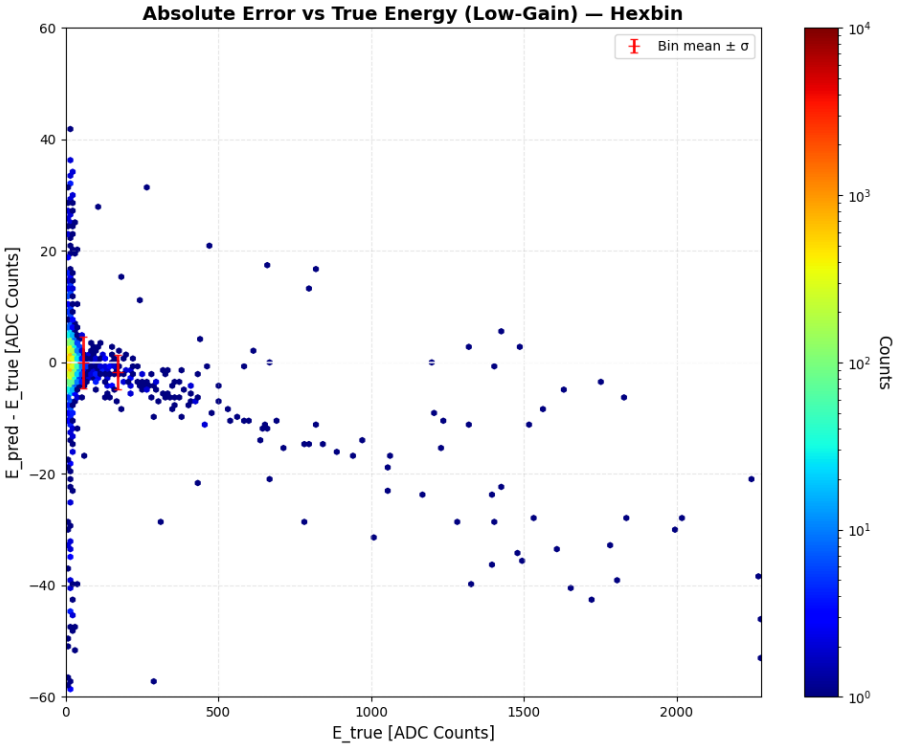

Figure 7: Absolute reconstruction error $|E_{\text{pred}} - E_{\text{true}}|$ vs. true LG energy for the Ridge Regression model. Mean absolute error (normalized): 0.0302.

7 Conclusion

A fully linear energy reconstruction model based on ridge regression was developed and trained on the Low-Gain channel of the ATLAS TileCal detector simulation dataset. The model takes the normalized 7-sample ADC pulse window as input and produces a scalar energy estimate via a single learned weighted sum, satisfying the strict linearity requirement for FPGA deployment in the ATLAS Level-0 hardware trigger.

The ridge regression model, trained with AdamW and L_2 regularization ($\lambda = 1 \times 10^{-3}$), achieves an R^2 of 0.9906 and a mean relative error of 0.41 on the held-out test set. The L_2 penalty suppresses large weights on noisy out-of-time pile-up samples, yielding robust generalization across the mid-to-high energy range. Elevated RMS relative error (10.05) is attributed to a small fraction of low-energy events where pile-up obscures the true signal, a known limitation shared with MLP and CNN models trained on identical data [1, 2].

References

[1] F. Curcio, *Machine Learning-Based Energy Reconstruction for the ATLAS Tile Calorimeter at HL-LHC.*

[2] D. Oliveira, *Energy Reconstruction of the ATLAS Tile Calorimeter under High Pile-up Conditions using the Wiener Filter.*